Structured Programming: C Fundamentals (Midterm Notes)

1. Problem Solving and Methodology (Week 1)

- **Steps:** Problem definition -> Analysis -> Solution design -> Implementation -> Testing/Evaluation.
- **Algorithm:** A finite sequence of well-defined steps to solve a problem.
- **Tools:**
    - Flowcharts: Graphical representation of logic.
    - Pseudocode: Structured, English-like description of the code.
- **Debugging:** Identifying and fixing errors (Syntax, Runtime, Logical).

2. Programming Language Fundamentals & Paradigms (Week 2)

- **Definition:** A system of communication used to direct computer activities.
- **Classification by Level:**
    - **Low-Level:** 1GL (Machine Language), 2GL (Assembly Language).
    - **High-Level:** User-friendly, English-like syntax (3GL: C, Pascal; 4GL: SQL).
- **Translators:**
    - Assembler: Assembly to machine code.
    - Compiler: Translates entire program at once; faster execution (C, C++).
    - Interpreter: Translates line-by-line; slower execution (Python).
- **Programming Paradigms:**
    - Structured: Structured control flows, no goto statements.
    - Procedural: Divided into procedures or functions (Top-down).
    - Object-Oriented (OOP): Objects (data + methods) (Bottom-up).

3. Introduction to C, Tokens & Data Types (Week 3)

- **History:** 1972 by Dennis Ritchie (Bell Labs) for UNIX.
- **Program Structure:** Preprocessor Directives, Main Function, Statements (end with ;).
- **Tokens:** Keywords, identifiers, constants, strings, operators.
- **Identifiers:** User-defined names that must start with a letter or _, and are case-sensitive.
- **Data Types:**
    - `int`: 4 bytes, `%d` (e.g., `int age = 20;`)

    - `float`: 4 bytes, `%f` (e.g., `float pi = 3.14;`)
    - `double`: 8 bytes, `%lf` (e.g., `double e = 2.718;`)
    - `char`: 1 byte, `%c` (e.g., `char grade = 'A';`)

4. Expressions, Operators, and I/O (Week 4)

- **Expression/Statement:** An expression terminated by a semicolon (;).
- **Operators:**
    - **Arithmetic:** `+`, `-`, `*`, `/`, `%` (modulus).
    - **Relational:** `==`, `!=`, `>`, `<`, `>=`, `<=`.
    - **Logical:** `&&` (AND), `||` (OR), `!` (NOT).
    - **Increment/Decrement:** `++` and `--`.
- **I/O Functions:**
    - `printf("Age: %d\n", age);` - Outputs to screen.
    - `scanf("%d", &age);` - Reads from keyboard.
- **Library Functions:** `<stdio.h>`, `<math.h>`, `<string.h>`.

5. Control Structures & Loops (Week 5)

- **Conditional Structures:**
    - **if-else:** Two paths based on a condition.
    - **switch:** Selects one of many blocks; cleaner than if-else chains.
- **Loops:**
    - **for Loop:** Used when exact iterations are known.
    - **while Loop:** Entry-controlled; used when iterations are unknown.
    - **do-while Loop:** Exit-controlled; executes at least once.

6. Functions (Week 5)

- **Definition:** A self-contained block used to perform a task.
- **Core Elements:**
    - **1. Prototype:** `int add(int a, int b);`
    - **2. Definition:** `int add(int a, int b) { return a + b; }`
    - **3. Call:** `int result = add(5, 3);`
- **Parameters vs. Arguments:** Parameters are in the declaration, Arguments are the passed values.
-